Modelado y análisis del flujo vehicular mediante teoría de colas para la optimización de tiempos de semáforo en intersecciones viales

Juan Calopino, Daniel Saavedra, Anthony Yaguana, Odalis Rosillo
Computación, Universidad Nacional de Loja
Loja, Ecuador
juan.calopino@unl.edu.ec, daniel.saavedra@unl.edu.ec , anthony.j.yaguana@unl.edu.ec, odalis.rosillo@unl.edu.ec

I. Tema

Modelado y análisis del flujo vehicular mediante teoría de colas para la optimización de tiempos de semáforo en intersecciones viales.

II. Resumen

El estudio presenta un modelo computacional de eventos discretos desarrollado en Python para analizar el flujo vehicular y la formación de colas en intersecciones semaforizadas. A través de la implementación de procesos de Poisson y colas FIFO, los autores evalúan cinco escenarios diferentes con métricas clave como la demora, la longitud máxima de las filas y el tiempo de espera bajo diferentes escenarios de demanda y duración de fases. Los resultados experimentales demuestran que ajustar los tiempos de luz verde reduce significativamente la congestión, aunque los autores señalan la necesidad de incorporar datos de campo reales y restricciones físicas reales adicionales para optimizar la representación del tráfico urbano.

III. Palabras clave

Teoría de colas, Flujo Vehicular, FIFO, Tràfico Urbano, Luz Verde, Congestiòn

IV. Introducción

A. Contexto

Las intersecciones semaforizadas son puntos relevantes de una red vial por qué los movimientos vehiculares en conflicto deben utilizar el espacio de cruce en momentos distintos. Durante la fase roja, los vehículos se acumulan; al comenzar la fase verde, la descarga puede tardar en alcanzar un ritmo estable debido al tiempo de reacción y aceleración de los conductores. Si el tiempo asignado a una fase no se corresponde con la demanda y la capacidad de descarga, parte de la fila puede permanecer al finalizar la fase verde y trasladarse al ciclo siguiente [1].

La teoría de colas permite representar la relación entre llegadas, servicio y espera, mientras que la simulación permite observar la evolución del sistema bajo supuestos explícitos. La utilidad del estudio depende de mantener la coherencia entre las ecuaciones, el algoritmo y las métricas. Por ello, este trabajo prioriza un modelo sencillo, reproducible y verificable antes de incorporar métodos de optimización más complejos [2].

B. Antecedentes

El análisis y la optimización del flujo vehicular en intersecciones semaforizadas han propiciado el desarrollo de una amplia variedad de metodologías dentro de la literatura especializada. La comunidad científica ha abordado los problemas de congestión, demoras y formación de colas desde diversas perspectivas, combinando modelos analíticos estocásticos basados en teoría de colas y procesos de Markov con formulaciones de optimización convexa, marcos de simulación microscópica multiobjetivo y sistemas adaptativos impulsados por inteligencia artificial.

Con el propósito de superar las deficiencias de los esquemas semafóricos rígidos que asumen tasas de evacuación constantes —omitiendo el retardo por arranque y la aceleración progresiva de los vehículos detenidos—, Elbaum, Novoselsky y Kagan [1] desarrollaron un modelo analítico fundado en la teoría de colas dependiente del estado. Su metodología representa cada carril del cruce mediante un sistema de colas alternadas $M/M_n/1$, en el que la tasa de descarga se incrementa de forma dinámica durante la fase verde y decae de manera continua a lo largo del periodo de despeje. A partir de esta formulación, los autores dedujeron ecuaciones en forma cerrada que permiten calcular la duración óptima de la fase verde y del ciclo completo en función directa de la demanda de entrada. Su trabajo concluye que ajustar la temporización semafórica a la tasa de descarga real de la cola estabiliza efectivamente el flujo vehicular y prevenir acumulaciones severas en el cruce.

Por su parte, Yang y Shi [2] abordaron la imprecisión que genera suponer arribos vehiculares individuales y aislados, una hipótesis convencional que subestima la magnitud real de las colas al ignorar que el tráfico urbano suele desplazarse en pelotones procedentes de semáforos contiguos. Para reflejar esta dinámica, estructuraron un modelo estocástico de colas con vacaciones del servidor (server vacations) y llegadas en lote (batch arrivals), resuelto mediante análisis matricial para sistemas de tipo M/G/1. Esta metodología les permitió derivar

la función de distribución de probabilidad estacionaria de la longitud de la cola a lo largo del ciclo semafórico, denominada Q(t). Los autores concluyeron que considerar las llegadas en grupo resulta indispensable para no subestimar la congestión y demostraron la validez teórica de conceptualizar la fase roja como vacaciones del servidor.

En el ámbito de la confiabilidad del tiempo de viaje, Mohajerpoor, Saberi y Ramezani [3] examinaron la principal limitación de los métodos clásicos de sincronización —como la fórmula de Webster—, los cuales se enfocan exclusivamente en minimizar la demora promedio e ignoran tanto la dispersión de los tiempos de espera como el riesgo de que las colas desborden la capacidad física del enlace (spillback). Para resolver este inconveniente, integraron la teoría de ondas de choque cinemáticas y derivaron analíticamente la función de densidad de probabilidad continua de la demora vehicular en condiciones no saturadas. Con base en esta función, formularon programas de optimización convexa que determinan configuraciones fijas y dinámicas enfocadas en reducir la varianza del tiempo de espera y la probabilidad de spillback. Sus hallazgos concluyen que este enfoque garantiza una asignación equitativa del tiempo de fase verde entre accesos principales y secundarios, incrementando la confiabilidad del sistema sin penalizar la demora global.

La problemática del control coordinado en intersecciones con capacidad de almacenamiento finita, donde la saturación del carril provoca el desvío o rechazo de vehículos (*balking*), fue estudiada por Castillo, Clempner y Poznyak [4]. Su propuesta metodológica recurre a cadenas de Markov de tiempo continuo modeladas como juegos no cooperativos, en los que cada controlador semafórico opera como un jugador independiente bajo una disciplina de atención FIFO. Mediante un algoritmo de gradiente proximal, los autores obtuvieron un equilibrio de Nash para definir los tiempos de conmutación de las fases luminosas. El trabajo concluye que esta representación matricial logra sincronizar con precisión los accesos en conflicto y minimizar la longitud esperada de las colas, todo ello sin requerir los elevados costos computacionales de los modelos macroscópicos de red.

Frente a los escenarios de congestión severa, demoras elevadas y emisiones contaminantes en zonas escolares con fuerte asimetría de tráfico durante las horas pico, Zhang y Sun [5] evaluaron las rigideces de los planes semafóricos de tiempos fijos y del método tradicional de Webster. Para dar respuesta a este entorno crítico, acoplaron la plataforma de simulación microscópica SUMO con el algoritmo genético de ordenamiento no dominado NSGA-II. Esta integración dio lugar a un esquema de temporización multiobjetivo de compromiso capaz de optimizar simultáneamente el tiempo de espera, la frecuencia de paradas y las emisiones de CO_2. Como conclusión principal, el estudio demostró que el plan optimizado supera a las fórmulas analíticas clásicas, reduciendo la demora media de 74.3 a $49.2 s/veh$ y la longitud promedio de la cola de 16.2 a 9.8 $vehículos$.

Finalmente, Errajarji, Bouayad y Fardousse [6] investigaron la falta de adaptabilidad de las señales de tráfico ante patrones de flujo heterogéneos y altamente variables, así como la lenta convergencia y la tendencia a caer en subóptimos locales que presentan los algoritmos convencionales de aprendizaje por refuerzo. Con el fin de superar estas trabas, diseñaron un sistema de control adaptativo inteligente que combina la optimización metaheurística por enjambre de salpas (*Salp Swarm Optimization*, SSO) con redes neuronales profundas (*Deep Q-Network*, DQN). En esta arquitectura, el algoritmo SSO orienta la fase de exploración del agente DQN durante su entrenamiento en entornos de simulación microscópica. Las conclusiones del estudio señalan que este modelo híbrido disminuye el tiempo acumulado de espera en hasta un 18% en comparación con las redes neuronales estándar, mostrando una notable capacidad de adaptación para regular de forma coordinada el paso de vehículos particulares, unidades de emergencia y peatones.

C. Problema y justificación

El problema que se estudiará es la posible acumulación de vehículos cuando la cantidad de vehículos que llega durante un ciclo supera la cantidad que el acceso puede atender. Una tasa de servicio constante puede simplificar demasiado el inicio de la fase verde; no obstante, introducir una tasa variable sin definir sus unidades, interpretación y procedimiento de estimación también puede producir conclusiones erróneas. Se requiere, por tanto, una formulación transparente y una simulación verificable que permita comparar escenarios sin asumir de antemano que una programación es óptima.

La propuesta es adecuada como proyecto computacional porque permite aplicar modelado, generación de variables aleatorias, programación orientada a eventos, análisis de datos y visualización. Su validez estará limitada por los supuestos adoptados y por la ausencia de datos locales de calibración, salvo que el equipo consiga aforos u observaciones de campo.

D. Propuesta

El modelo que se propone consiste en representar un acceso con semáforo como si fuera la cola de un servidor, combinando llegadas que ocurren al azar y un servicio que se activa de forma periódica. Mientras el semáforo está en rojo, los autos que van llegando se van acumulando en la fila, ya que no pueden avanzar. Al cambiar a verde, los vehículos empiezan a salir respetando una tasa de servicio fija que ya se

estableció de antemano. Todo esto se llevará a la práctica en Python a través de una simulación de eventos discretos, lo que nos dará la oportunidad de ir registrando las llegadas, las salidas, en qué estado se encuentra el semáforo y cómo va cambiando el tamaño de la cola conforme pasa el tiempo.

Para comprobar que el simulador funciona correctamente, se van a hacer pruebas probando distintas tasas de llegada y varios ajustes en los tiempos que duran las luces rojas y verdes. Dependiendo del escenario, se calcularán ciertos indicadores, como por ejemplo la longitud promedio y la máxima de la cola, el tiempo de demora que experimenta cada vehículo en promedio, y cuántos autos se quedan esperando todavía cuando la fase verde termina. Para que la aleatoriedad afecte lo menos posible y los resultados sean confiables, se usarán varias réplicas de la simulación para poder compararlas.

V. Objetivos

A. Objetivo general

Modelar y analizar mediante simulación el comportamiento de una cola vehicular en un acceso semaforizado, considerando llegadas estocásticas y servicio condicionado por las fases del semáforo, para evaluar cómo la demanda y los tiempos de fase afectan la longitud de cola y la demora.

B. Objetivos específicos

- Definir las variables, unidades, supuestos y reglas de llegada y servicio que conforman el modelo de colas vehiculares.
- Implementar una simulación de eventos discretos mediante un lenguaje de programación que permita registrar las llegadas, las salidas, el estado del semáforo y la evolución de la cola vehicular.
- Analizar el comportamiento de la cola mediante la comparación de escenarios de demanda y configuraciones semafóricas, a través de métricas estándar y simulaciones controladas, considerando las variaciones respecto al tráfico real.

VI. Metodología

El proyecto se desarrolló a partir de la investigación de información en bases de datos confiables como IEEE Xplore y ACM Digital Library. Luego se sintetizó la información para identificar los conceptos clave necesarios para el proyecto y, finalmente, se desarrolló siguiendo las 7 etapas de un proyecto de simulación:

- Definición del problema
- Formulación conceptual
- Recolección y análisis de datos
- Construcción del modelo
- Verificación y validación
- Experimentación y análisis de resultados
- Implementación y documentación.

A. Enfoque

Se emplea un enfoque cuantitativo y computacional, porque las respuestas del modelo se evalúan mediante indicadores numéricos obtenidos de varias réplicas independientes.

B. Tipo de estudio

El estudio es experimental y comparativo, basado en simulación. Se modifican de manera controlada la tasa de llegada y los tiempos de las fases del semáforo, mientras se mantienen constantes los demás parámetros del modelo, y se comparan los escenarios resultantes bajo condiciones controladas.

C. Herramientas y recursos

- **Lenguaje de programación:** Python 3.
- **Frameworks o bibliotecas:** numpy (generación de llegadas aleatorias), scipy (intervalos de confianza con la distribución t de Student), matplotlib (gráficas y animación), Pillow (exportar la animación como GIF, opcional), pytest (pruebas automáticas) y Tkinter (interfaz gráfica).
- **Software:** Python 3.10 o superior (el entorno virtual del proyecto usa Python 3.13) con las dependencias de requirements.txt.
- **Conjunto de datos:** no se utilizan datos de campo. Las llegadas se generan en la simulación y los parámetros son supuestos experimentales (véase la Etapa 3).
- **Modelos matemáticos utilizados:** proceso de Poisson homogéneo para las llegadas (tiempos entre llegadas exponenciales con media $1/\lambda$), servicio determinista a tasa μ durante la fase verde, disciplina FIFO y semáforo con fases roja y verde de duración fija, implementados mediante una simulación de eventos discretos con agenda ordenada cronológicamente. La dinámica de la cola se presenta en la sección Modelo Matemático.
- **Fuentes de información:** IEEE Xplore y ACM Digital Library.

D. Procedimiento

Etapa 1: Definición del problema

Se identifica como problema la posible acumulación de vehículos cuando la cantidad que llega durante un ciclo supera la que el acceso puede atender.

Con la información investigada y sintetizada se ha seleccionado una simulación computacional de eventos discretos de un acceso vehicular controlado por un semáforo. El modelo permitirá estudiar cómo la tasa de llegada de vehículos y la distribución del tiempo del ciclo entre las fases roja y verde afectan la evolución de la cola, la espera y la cantidad de vehículos atendidos. El alcance inicial se limita a un acceso y una cola; por ello, los resultados no representan por sí solos el comportamiento de una intersección completa con varios movimientos.

Etapa 2: Formulación conceptual

El sistema real se representa con los siguientes elementos: un único acceso semaforizado, una sola cola, un servidor y un semáforo que alterna fases roja y verde.

Variables de entrada:

1) Tasa de llegada (λ), expresada en vehículos por segundo (veh/s).
2) Duración de la fase verde (g), en segundos.
3) Duración de la fase roja (r), en segundos.
4) Duración del ciclo (T = g + r), en segundos.
5) Tasa de servicio durante el verde (μ), en vehículos por segundo (veh/s).
6) Horizonte de simulación (H), en segundos.
7) Número de réplicas (n) y semilla aleatoria asignada a cada réplica.

Variables de estado:

- Número de vehículos que esperan en la cola, Q(t).
- Estado del semáforo: rojo o verde.
- Estado del servidor: libre u ocupado.
- Vehículo que está siendo atendido, si corresponde.
- Tiempo del evento actual e índice del ciclo semafórico.

Variables de salida:

1) Longitud media temporal de la cola y longitud máxima de la cola.
2) Tiempo medio de espera de los vehículos que iniciaron el servicio y, por separado, tiempo medio en el sistema de los vehículos que salieron.
3) Número de vehículos atendidos por ciclo.
4) Cola residual al finalizar cada fase verde.
5) Número de vehículos pendientes al terminar el horizonte.
6) Desviación estándar e intervalo de confianza del 95 % de las métricas principales entre réplicas.

Supuestos iniciales y reglas del modelo:

- Se representa un único acceso semaforizado con una sola cola y un servidor. No se modelan otros accesos, giros, peatones ni conflictos entre movimientos.
- Las llegadas siguen un proceso de Poisson homogéneo; por tanto, los tiempos entre llegadas son exponenciales con media $1/\lambda$. Si $\lambda = 0$, no se generan llegadas.
- La disciplina de atención es FIFO: el primer vehículo que llega es el primero en recibir el servicio.
- El semáforo alterna fases roja y verde de duración fija. El ciclo se calcula como $T = g + r$; no se implementa control adaptativo.
- Durante la fase roja no se atienden vehículos. Durante la fase verde, el servidor atiende a una tasa determinista μ, equivalente a un vehículo cada $1/\mu$ segundos.
- Un servicio solo comienza si puede finalizar antes o exactamente al final de la fase verde. Los vehículos que no alcanzan a ser atendidos permanecen en espera para el siguiente verde.
- No se incluye fase amarilla ni intervalo de despeje. Esta simplificación debe mencionarse al interpretar los resultados.
- La cola tiene capacidad ilimitada. Por ello, no se calcula desbordamiento físico del acceso; este indicador solo podrá incorporarse si se define la longitud útil de almacenamiento y el espacio medio ocupado por vehículo.
- La simulación comienza al inicio de una fase roja, con cola vacía, y no contempla período de calentamiento. Los resultados corresponden al horizonte observado, especialmente en escenarios cercanos a la saturación o sobresaturados.
- Los parámetros de llegada y servicio son supuestos experimentales mientras no se calibren con observaciones de campo o fuentes técnicas justificadas.

Etapa 3: Recolección y análisis de datos

No se dispone de datos reales observados de una intersección. Los datos necesarios para el modelo (tiempos, cantidades, frecuencias y probabilidades) se definen como datos sintéticos:

- **Tasa de servicio:** $\mu = 0.5$ veh/s (un vehículo cada dos segundos de verde), equivalente a una capacidad nominal de 1800 veh/h durante el verde. Este valor es provisional y deberá justificarse con una fuente técnica o calibrarse con datos de campo antes de interpretarlo como representativo de una intersección real.

- **Horizonte y réplicas:** H = 3600 s y n = 30 réplicas independientes por escenario. Todos los parámetros se editan en config/config.json sin modificar el código.
- **Llegadas:** generadas mediante un proceso de Poisson de tasa λ (tiempos entre llegadas exponenciales).
- **Semillas:** la réplica y usa la semilla 2025 + i (semilla_base = 2025 en config.json), es decir, las semillas 2026 a 2055.

Los escenarios iniciales son los siguientes:

Tabla I: Escenario Inicial

Escenario	λ (veh/s)	g (s)	r (s)	ρef = λT/(μg)
Base	0.15	30	30	0.60
Verde corto	0.15	20	40	0.90
Verde largo	0.15	40	20	0.45
Demanda alta	0.25	30	30	1.00
Sobresaturado	0.35	30	30	1.40

El indicador ρef se utiliza como cociente de la demanda esperada por ciclo respecto a la capacidad nominal de servicio durante el verde; no debe interpretarse como garantía de equilibrio estacionario. La capacidad nominal por ciclo se calcula como μg y las llegadas esperadas como λT.

Para una validación empírica posterior será necesario obtener datos de campo, como volúmenes de llegada por intervalo, tiempos de fase, tasas de descarga, longitudes de cola y demoras observadas. Con esos datos se podrán calibrar los parámetros y comparar las métricas simuladas con las observadas.

Etapa 4: Construcción del modelo

El modelo conceptual se transforma en un simulador implementado en Python 3 mediante una agenda de eventos discretos ordenada cronológicamente. El programa separa modelo, controlador y vista. cada uno con una función distinta, es decir el model no imprime, ni gráfica, el controlador ejecuta escenarios, réplicas, pruebas y el barrido de g. y las vistas muestran tablas, gráficas, animación e interfaz gráfica, además exportan los CSV. La agenda de eventos es un heap binario ordenado por tiempo, prioridad y secuencia.

- Configurar el horizonte H, λ, g, r, μ, el número de réplicas y las semillas. Verificar que $\lambda \geq 0$ y que g, r, μ y H sean positivos; calcular T = g + r.
- Inicializar el sistema al comienzo de la fase roja, con cola vacía y servidor libre.
- Generar las llegadas mediante tiempos entre llegadas exponenciales. Cada llegada se registra con identificador y tiempo de llegada, y programa la llegada siguiente cuando corresponda.
- Programar los cambios de fase roja-verde y verde-roja, y procesar llegadas, salidas y cambios de fase en orden cronológico hasta H.
- En cada llegada, incorporar el vehículo a la cola de espera y registrar el evento y el tamaño de la cola.
- Al comenzar o continuar una fase verde, si hay vehículos esperando y el servidor está libre, programar una salida en t + 1/μ solo si esa salida ocurre antes o exactamente al final del verde.
- En cada salida, registrar el tiempo de salida y la espera del vehículo, retirar al vehículo del sistema, liberar el servidor y comprobar si puede atenderse al siguiente vehículo durante el verde restante.
- Al finalizar cada verde, registrar los vehículos atendidos en el ciclo y la cola residual. Al alcanzar H, cerrar el cálculo del área bajo Q(t) y registrar todos los vehículos que siguen pendientes.

Si coinciden eventos en el mismo instante, se aplica la prioridad salida, luego cambio de fase y luego llegada; entre eventos del mismo tipo se respeta el orden en que se programaron. Una salida que termina justo al final del verde es válida y se cuenta en ese ciclo.

Etapa 5: Verificación

El estudio distingue dos actividades en esta etapa: la verificación, para comprobar que el programa implementa las reglas definidas, y la validación, para valorar la correspondencia del modelo con el tráfico real.

Verificación. Antes del experimento principal, se ejecutan 9 pruebas automáticas (con pytest o desde el menú del programa); todas pasan:

- Con λ = 0, no se generan llegadas ni salidas y la cola permanece vacía.
- Ninguna salida ocurre durante la fase roja.
- La cantidad de vehículos llegados coincide con los que salieron más los que permanecen en el sistema al finalizar la corrida.
- Una misma configuración y semilla producen la misma traza de eventos y las mismas métricas; con la misma semilla y λ, las llegadas son idénticas aunque cambien g y r.

- Con demanda saturante, las salidas por verde no superan μg y son exactamente μg cuando g es múltiplo de 1/μ.
- Los eventos están en orden cronológico y no hay salidas después del horizonte H.
- En el escenario sobresaturado, la cola residual crece de los primeros a los últimos ciclos (10 semillas).
- Se respeta la prioridad de eventos simultáneos: salida, cambio de fase y llegada.

Una novena prueba comprueba que las entradas fuera de dominio producen un error claro. Estas pruebas verifican la consistencia interna del programa, pero no prueban que los parámetros representen adecuadamente el tráfico real.

Etapa 6: Experimentación y análisis de resultados

Se ejecutan los escenarios de la Tabla 1 y se analizan los resultados obtenidos mediante las métricas definidas en la sección de evaluación

Además se ejecuta un barrido de g con T = 60 s y λ = 0.15 veh/s, para g = 10, 15, …, 50 s (r = T − g), que produce la Tabla I y la Fig. 2.

- Cada escenario se repite con semillas registradas, y los resultados se comparan mediante las métricas definidas.
- Para estudiar el efecto de los tiempos de fase, se comparan configuraciones con la misma demanda. Cuando T se mantiene fijo, variar g implica variar r; por tanto, el experimento evalúa la redistribución del tiempo del ciclo. Si también se desea estudiar el efecto de la duración total del ciclo, deberá diseñarse un conjunto adicional de escenarios donde T cambie de manera explícita.
- Para comparar configuraciones se usan números aleatorios comunes: cada réplica crea un generador de numpy dedicado solo a los intervalos entre llegadas, de modo que, con la misma semilla y λ, el flujo de llegadas es idéntico aunque cambien g o r. Así las diferencias entre configuraciones reflejan los tiempos de fase y no el ruido de muestreo.
- Para cada indicador se reporta la media entre réplicas, la desviación estándar muestral y un intervalo de confianza del 95 % basado en la distribución t de Student.
- Las reducciones porcentuales se calculan respecto de la configuración base bajo la misma tasa de llegada, indicando claramente qué indicador se compara.
- En escenarios cercanos o superiores a la capacidad nominal, los resultados se interpretan como resultados del horizonte simulado y no como estimaciones de régimen estacionario.

Etapa 7: Implementación y documentación

En esta etapa se realiza la implementación del modelo y de la simulación en código, acompañada de la documentación formal expresada en este documento, el cual detalla la investigación y la metodología empleada.

A continuación se presenta el link del repositorio donde se encuentra la implementación del código:

https://github.com/odalis-rg/SimulacionTraficoInterseccion.git

E. Evaluación

La longitud media temporal de la cola se calcula como el área bajo Q(t) dividida por el horizonte H:

$$\bar{Q} = (1/H) \int_0^H Q(t)\, dt$$

Como Q(t) permanece constante entre eventos, el área se calcula sumando el producto del tamaño de la cola por la duración de cada intervalo entre eventos. Se incluye el intervalo final hasta H.

La longitud máxima es el mayor valor observado de Q(t) durante el horizonte. La cola residual de cada ciclo es el número de vehículos que siguen esperando al finalizar la fase verde.

Se distinguen dos medidas de tiempo. El tiempo de espera de un vehículo es el instante en que comienza su servicio menos su instante de llegada. El tiempo en el sistema es el instante de salida menos el instante de llegada. Si el programa no registra el inicio del servicio, no podrá calcular directamente la espera pura; en ese caso, deberá reportar el tiempo en el sistema y evitar denominarlo espera media.

Para los vehículos que no hayan salido al terminar H, se reporta su cantidad y, de manera complementaria, el tiempo transcurrido desde su llegada hasta H. Este último valor se identifica como observación censurada y no se mezcla sin aclaración con la media de los vehículos que sí salieron. Así se evita que las métricas aparenten ser mejores por excluir vehículos con esperas largas.

También se contabilizan las salidas válidas por ciclo y se comprueba el balance de vehículos: vehículos llegados = vehículos que salieron + vehículos que permanecen en el sistema, considerando la definición temporal y los eventos procesados hasta H.

Los criterios de evaluación se relacionan con los objetivos específicos de la siguiente manera:

Tabla II: Criterios de Evaluación

Objetivo específico	Criterio de evaluación
1. Definición de variables, unidades, supuestos y reglas	Coherencia de la definición del modelo (Etapa 2)
2. Implementación de la simulación de eventos discretos	Pruebas de verificación y balance de vehículos (Etapas 4 y 5)
3. Análisis de la cola en escenarios de demanda y configuraciones semafóricas	Métricas de esta sección, media, desviación estándar e IC 95 % entre réplicas (Etapa 6)

VII. Resultados

A. Resultados del objetivo específico 1

El sistema modelado es un único acceso semaforizado con una cola FIFO de capacidad ilimitada y un servidor. El servidor representa la línea de detención y solo opera durante el verde.

Tabla III: *Variables, unidades y valores adoptados*

Variable	Símbolo	Unidad	Valor o rango
Tasa de llegada	λ	veh/s	0.15; 0.25; 0.35
Tasa de servicio en verde	μ	veh/s	0.5 (1 vehículo cada 2 s)
Duración del verde	g	s	20, 30, 40 (escenarios); 10 a 50 (barrido)
Duración del rojo	r	s	20, 30, 40 (escenarios); T − g (barrido)
Ciclo	T = g + r	s	60
Horizonte de simulación	H	s	3 600
Réplicas por escenario	n	—	30 (semillas 2026 a 2055)
Nivel de confianza	—	—	95 % (t de Student)
Intensidad efectiva	ρef = λT/(μg)	—	0.36 a 1.80

Reglas de llegada y servicio

- Las llegadas siguen un proceso de Poisson homogéneo, con intervalos exponenciales de media 1/λ.
- El servicio es determinista y dura 1/μ = 2 s por vehículo.
- En rojo no se atiende a nadie.
- En verde, un servicio solo comienza si termina antes o exactamente al final del verde. Por eso salen como máximo μ·g vehículos por verde (15 con g = 30 s).
- Los eventos simultáneos se procesan en el orden de salida, cambio de fase y llegada.

Supuestos

- El semáforo tiene solo dos fases (rojo y verde), sin amarillo ni tiempo perdido al arrancar.
- La simulación empieza en rojo, con la cola vacía y sin período de calentamiento.
- No hay giros, peatones, pelotones ni otros accesos.

B. Resultados del objetivo específico 2

El simulador se programó en Python y con ayuda de la IA con arquitectura modelo-vista-controlador:

```
Simulación de un acceso semaforizado — menú principal
  1) Ejecutar pruebas de verificación
  2) Ejecutar un escenario
  3) Ejecutar todos los escenarios
  4) Ejecutar el barrido de g (Tabla I y Fig. 2)
  5) Ver la traza de eventos de una réplica
  6) Ver la animación de la simulación de un escenario
  7) Ver la configuración activa
  8) Abrir la interfaz gráfica (escenarios y gráficas)
  0) Salir
Opción: 8
```

Fig. 1. *Menú de la consola.*

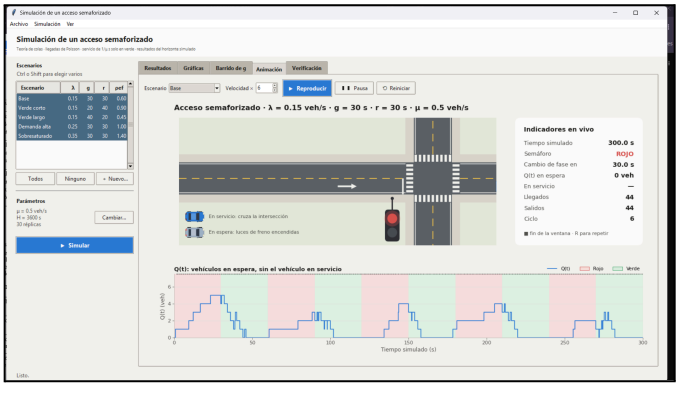


Fig. 2. *Interfaz gráfica del sistema.*

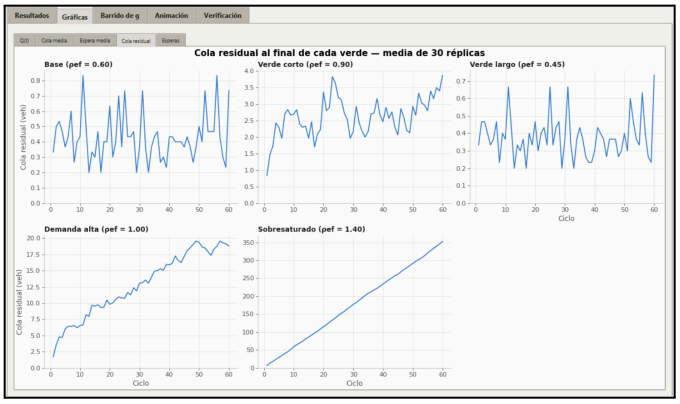


Fig. 3. *Gráficas*

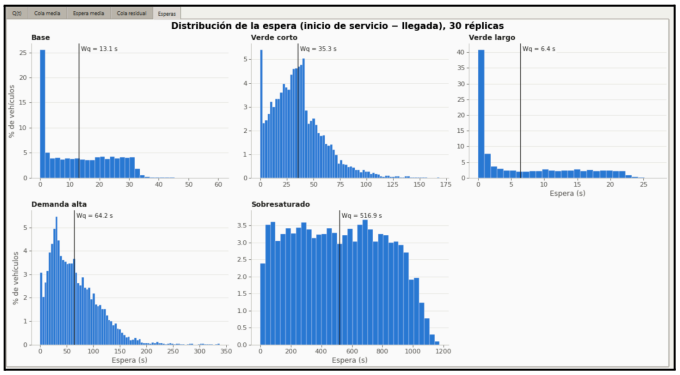


Fig. 4. *Gráficas*

- **Modelo:** vehículo, semáforo, agenda de eventos (un heap), motor de simulación y estadísticas.
- **Vista:** consola, interfaz gráfica, gráficas, animación y exportación a CSV.
- **Controlador:** escenarios, réplicas, semillas y barrido.

El motor registra las llegadas, las salidas y los cambios de fase. En cada evento guarda el instante, el estado del semáforo, la longitud de la cola Q(t), el vehículo en servicio y el ciclo. En la réplica 1 del escenario base se muestra el funcionamiento:

Tabla IV: Resultados del Escenario Base

Tiempo (s)	Evento	Vehículo	Semáforo	Q(t)	En servicio
0.992	Llegada	#1	Rojo	1	—
23.383	Llegada	#5	Rojo	5	—
30.000	Cambio de fase	—	Verde	4	#1
32.000	Salida	#1	Verde	4	#2
34.000	Salida	#2	Verde	3	#3
44.000	Salida	#7	Verde	0	#8
51.503	Salida	#10	Verde	0	—
60.000	Cambio de fase	—	Rojo	0	—

En el primer rojo se acumulan 5 vehículos. Al pasar a verde (30 s) salen uno cada 2 s y la cola se vacía hacia los 44 s. Al cerrar la hora, la réplica terminó con 504 vehículos llegados, 504 salidos, 0 en cola y 0 en servicio.

C. Resultado del objetivo específico 3

Escenarios: resultados promedio de 30 réplicas

Tabla V: Resultados por Escenario

Escenario	λ (veh/s)	g/r (s)	ρef	$\bar{Q}$ (veh)	Wq (s)	C (veh/h)
Base	0.15	30/30	0.60	1.95	13.10	534.87
Verde corto	0.15	20/40	0.90	5.27	35.29	531.73
Verde largo	0.15	40/20	0.45	0.95	6.42	534.87
Demanda alta	0.25	30/30	1.00	15.98	64.18	873.77
Sobresaturado	0.35	30/30	1.40	178.41	516.88	899.73

Impacto de los tiempos del semáforo y la cantidad de vehículos

- Verde corto (20 s): Se acumulan más vehículos y la espera aumenta de 13 a 35 segundos.
- Verde largo (40 s): La cola se reduce a la mitad y los vehículos esperan menos, aproximadamente 6 segundos.
- Demanda alta: Llegan más vehículos de los que el semáforo puede atender fácilmente. La cola aumenta a 16 vehículos y la espera a 64 segundos.
- Sobresaturado: Llegan 1 252 vehículos, pero solo salen 900. Quedan aproximadamente 352 vehículos esperando y la demora llega a 8.6 minutos.
- Capacidad máxima: Con el semáforo en verde durante 30 segundos, pueden pasar hasta 15 vehículos por ciclo.

- Cambios en el verde: Con 10 segundos de verde pasan aproximadamente 300 vehículos por hora; con 50 segundos pasan aproximadamente 535.
- Conclusión: Cuando el semáforo tiene más tiempo en verde, los vehículos pasan más rápido y se acumulan menos. Pero si llegan demasiados vehículos, la cola sigue creciendo.

Limitaciones frente al tráfico real

- No se utilizaron datos reales de vehículos; se trabajó con valores estimados.
- Solo se simuló un semáforo, sin considerar peatones, giros ni otros semáforos.
- No se incluyó la luz amarilla.
- La simulación comienza con la calle vacía y dura una hora.
- Los 50 segundos de verde dieron el mejor resultado en las pruebas, pero no significa que sean los mejores para una calle real.

VIII. Discusión

- *Variación de la fase verde:* La ampliación del verde a 40 s redujo la intensidad efectiva a 0.45, la cola media a 0.95 vehículos y la espera a 6.42 s. En cambio, el verde corto de 20 s elevó estos valores a 0.90, 5.27 vehículos y 35.29 s. Estos resultados coinciden con Elbaum et al., quienes relacionan la capacidad de servicio con la duración del verde. Sin embargo, nuestro modelo utiliza una tasa de servicio constante, mientras que el suyo considera la aceleración inicial de los vehículos. Aunque el verde largo mejora el flujo, no significa que sea óptimo para una intersección real, ya que podría perjudicar a los accesos transversales.
- *Incremento de la demanda vehicular:* Al aumentar la demanda de 0.15 a 0.25 veh/s, la intensidad efectiva alcanzó 1.00, con una cola de 15.98 vehículos y una espera de 64.18 s. En sobresaturación, con 0.35 veh/s, la cola llegó a 178.41 vehículos y la espera a 516.88 s. Estos resultados coinciden con Yang y Shi respecto al crecimiento de las colas bajo alta demanda. No obstante, nuestro modelo utiliza llegadas individuales de Poisson, mientras que ellos consideran llegadas en grupos. El simulador permite analizar la acumulación de vehículos en condiciones críticas, aunque todavía necesita incorporar llegadas agrupadas para representar mejor el tráfico real.
- *Relación entre intensidad de tráfico y demora:* Al pasar del escenario base, con intensidad efectiva de 0.60 y espera de 13.10 s, al verde corto, con intensidad de 0.90 y espera de 35.29 s, se observó que pequeños incrementos en la intensidad pueden aumentar considerablemente la demora. Esto coincide con Mohajerpoor et al., quienes analizan el comportamiento de las colas y su propagación. Sin embargo, nuestro modelo considera un carril de longitud ilimitada y no contempla el desbordamiento de las colas. Por ello, aunque permite evaluar la sensibilidad del tráfico, todavía no representa completamente las restricciones físicas de una intersección real.
- *Comportamiento de las colas en las réplicas:* En las 30 réplicas de Monte Carlo, el modelo FIFO procesó la demanda sin pérdidas de vehículos, alcanzando una cola promedio de 178.41 vehículos en sobresaturación. Esto difiere de Castillo et al., quienes consideran la capacidad limitada de las vías y el posible rechazo o desvío de vehículos. Nuestro modelo permite analizar el crecimiento de las colas de forma sencilla, pero no contempla restricciones físicas ni decisiones de los conductores. Por ello, incorporar una capacidad máxima y reglas de desvío permitiría representar mejor las condiciones reales del tráfico.
- *Barrido paramétrico de la fase verde:* El barrido de tiempos de verde entre 10 y 50 s mostró que asignar 50 s permitió obtener la menor cola, de 0.5 vehículos, y reducir la demora en el acceso evaluado. Estos resultados coinciden con Zhang y Sun y Errajarji et al., quienes muestran que ajustar los tiempos de verde puede mejorar el flujo vehicular. Sin embargo, nuestro modelo analiza un único acceso, mientras que sus estudios consideran múltiples movimientos y factores como las emisiones. El simulador permite evaluar diferentes tiempos de verde de manera sencilla, aunque todavía no contempla control adaptativo ni las condiciones de los accesos transversales.

IX. Conclusiones

- Se definió un modelo de colas vehiculares con llegadas aleatorias y un semáforo que permite el paso de vehículos únicamente durante la luz verde. Esto permitió establecer las reglas de funcionamiento y las variables necesarias para analizar la cola y los tiempos de espera.
- Se implementó el simulador en Python y se comprobó su funcionamiento mediante nueve pruebas automáticas, todas superadas. Además, el uso de semillas y 30 réplicas por escenario permitió repetir los experimentos y comparar los resultados de manera confiable.
- Se comprobó que aumentar el tiempo de luz verde reduce la cola y la espera, mientras que una mayor cantidad de vehículos puede provocar acumulaciones importantes. En el escenario sobresaturado quedaron aproximadamente 352 vehículos sin atender. Sin embargo, los resultados corresponden al modelo simulado y se necesitan datos reales para determinar los tiempos adecuados en una intersección vial.

X. Recomendaciones

- Se recomienda incorporar más factores al modelo, como la luz amarilla, los giros y el paso de peatones, para representar mejor el funcionamiento del tráfico real.
- Se recomienda realizar más pruebas con diferentes valores y comparar los resultados con datos reales de vehículos, para comprobar si el simulador representa correctamente el comportamiento del tráfico.
- Se recomienda probar diferentes tiempos de luz verde y distintos niveles de tráfico para encontrar mejores resultados. También se sugiere estudiar varios semáforos conectados, ya que aumentar el tiempo de verde en una vía puede afectar a las demás.

XI. Referencias bibliográficas

[1] A. Hajbabaie and R. F. Benekohal, "Traffic signal timing optimization," *Transp. Res. Rec.*, no. 2355, pp. 10–19, 2013, doi: 10.3141/2355-02.

[2] K. Khalid, A. Bouayad, and K. Fardousse, "Adaptive traffic lights control system using deep Q-network and salp swarm optimization," *IEEE Access*, vol. 13, pp. 114199–114207, 2025, doi: 10.1109/ACCESS.2025.3583656.

[3] R. Mohajerpoor, M. Saberi, and M. Ramezani, "Analytical derivation of the optimal traffic signal timing: Minimizing delay variability and spillback probability for undersaturated intersections," *Transp. Res. B, Methodol.*, vol. 119, pp. 45–68, Jan. 2019, doi: 10.1016/j.trb.2018.11.004.

[4] Y. Elbaum, A. Novoselsky, and E. Kagan, "A queueing model for traffic flow control in the road intersection," *Mathematics*, vol. 10, no. 21, Nov. 2022, doi: 10.3390/math10213997.

[5] Q. Yang and Z. Shi, "The evolution process of queues at signalized intersections under batch arrivals," *Physica A, Stat. Mech. Appl.*, vol. 505, pp. 413–425, Sep. 2018, doi: 10.1016/j.physa.2018.03.059.

[6] R. Castillo González, J. B. Clempner, and A. S. Poznyak, "Solving traffic queues at controlled-signalized intersections in continuous-time Markov games," *Math. Comput. Simul.*, vol. 166, pp. 283–297, Dec. 2019, doi: 10.1016/j.matcom.2019.06.002.

XII. Declaración de uso de la Inteligencia Artificial

Herramientas utilizadas: Claude y NotebookLM.

Forma de uso: NotebookLM se empleó para resumir los trabajos científicos recopilados e identificar sus ideas principales, facilitando su comparación con los resultados obtenidos en la investigación.

Prompts utilizados:

NotebookLM:

"Resume los trabajos científicos cargados, identifica sus principales aportes, metodologías y conclusiones, y destaca la información relevante para relacionarla con el tema de investigación".

"Redacta una comparación de los resultados de la simulación utilizando los trabajos científicos de la introducción. Compara los hallazgos, explica las posibles causas de los comportamientos observados e identifica los aportes y limitaciones del modelo".

El estudiante mantiene la responsabilidad de revisar y verificar la información generada, las referencias bibliográficas, los resultados y las conclusiones incluidas en el informe.

Claude:

TAREA:

Implementa en Python 3.10+ las capas Controlador y Vista del siguiente proyecto de simulación MVC de eventos discretos de un acceso semaforizado. El Modelo ya existe y está verificado: intégralo sin modificarlo.

SUPUESTOS DEL MODELO:

- llegadas de Poisson y cola FIFO ilimitada;
- servicio de 1/μ s solo en verde;
- semáforo fijo que inicia en rojo;
- Q(t) no cuenta el vehículo en servicio.

RESTRICCIONES:

- MVC estricto con SOLID: una responsabilidad por clase, inyección por constructor.
- Código en español.
- Algunas dependencias: numpy, scipy, matplotlib, pytest y Tkinter; Pillow opcional.
- No añadas el color amarillo al semaforo, solo se trabaja con verde y rojo.

VISTA:

- Consola: tablas de indicadores, ciclos, comparació.
- Exportación a CSV y gráficas PNG, histograma de esperas.
- Animaciones:
  - intersección vista desde arriba, autos, semáforo en tiempo real, indicadores en vivo.
  - botón de inicio, pausa y reinicio.

- Interfaz Tkinter:
    - panel de escenarios y parámetros.
    - pestañas Resultados, Gráficas, Barrido de g, Animación y Verificación;

ACEPTACIÓN

- Con la misma semilla, resultados idénticos a los del modelo.
- Comentarios explicativos en el código.
- Archivo requiriments.txt
- Interfaz sencilla y fácil de usar.